

Deploying Models with FastAPI

Lightweight, Pythonic serving for ML/AI

A modern approach to serving your ML models with minimal overhead and maximum flexibility.



API

Prediction

0.2040m

0.2040m

0.2045m

0.2055m

Prediction

0.2043m

0.2060m

0.2045m

Why FastAPI?

Modern Python Framework

Async by design, leveraging the latest Python features

High Performance

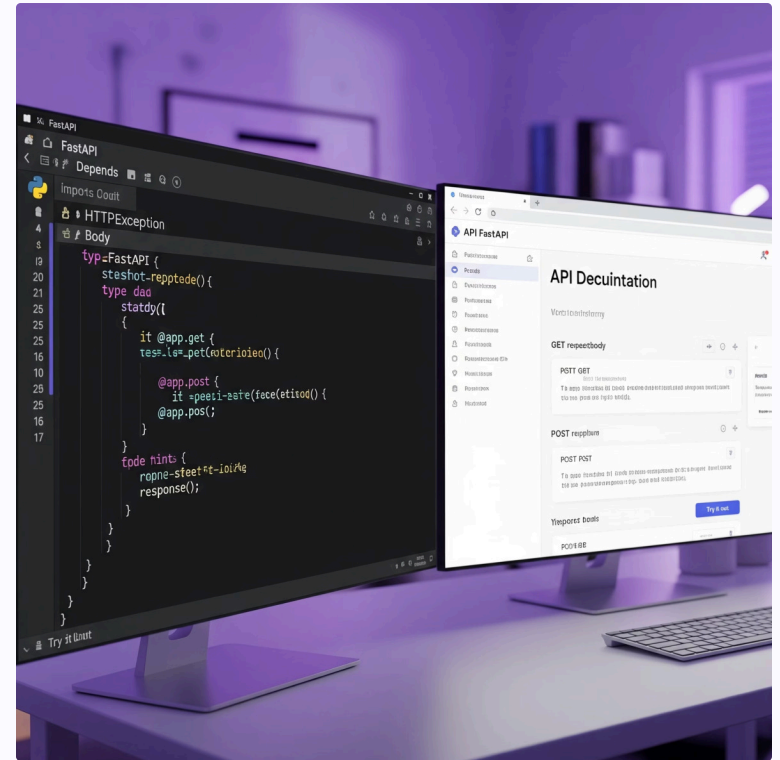
Built on Starlette and uvicorn for blazing fast response times

ML Library Integration

Seamlessly works with PyTorch, TensorFlow, and scikit-learn

Built-in OpenAPI Docs

Automatic documentation, type checking, and validation



Core Workflow



Load Your Trained Model

Import your PyTorch, TensorFlow, ONNX or other trained model



Create FastAPI App

Set up your FastAPI application with inference endpoints



Add Pre/Post Processing

Implement data transformation logic before and after model inference



Deploy

Run with uvicorn or containerize for production deployment

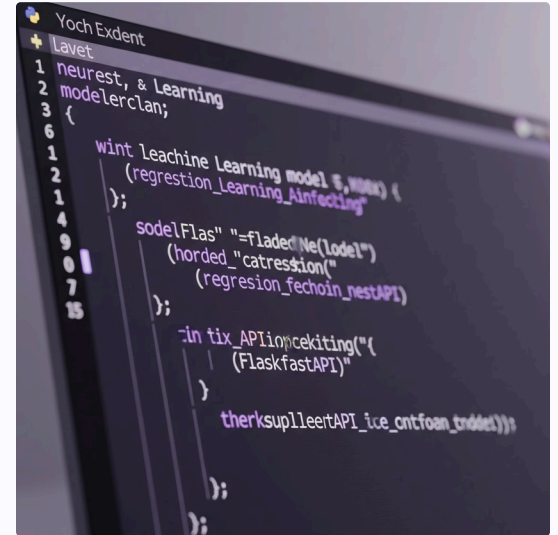
Minimal Example

```
from fastapi import FastAPI
import joblib

app = FastAPI()
model = joblib.load("model.pkl")

@app.post("/predict")
def predict(features: dict):
    return {"prediction": model.predict(
        [list(features.values())])[0]}
```

Run: `uvicorn main:app --host 0.0.0.0 --port 8000`



This minimal example loads a scikit-learn model and creates a prediction endpoint that takes feature values as input.

Adding Pydantic Models

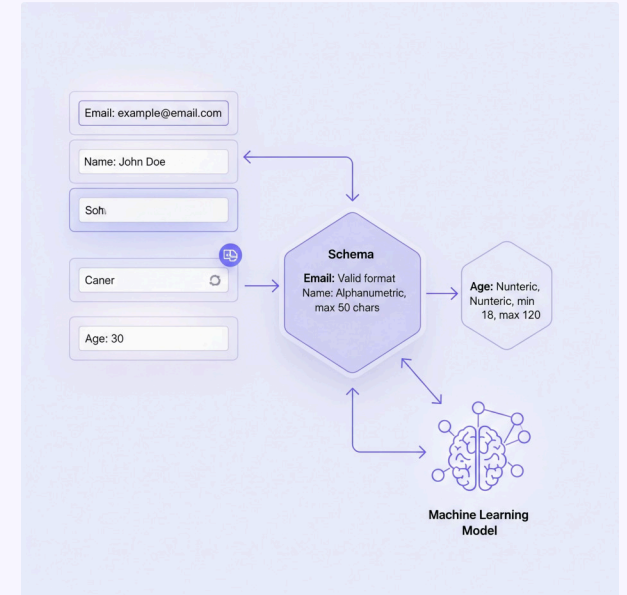
Use `BaseModel` for request/response schemas to ensure validation and enhance API docs

```
from fastapi import FastAPI
from pydantic import BaseModel

class Input(BaseModel):
    age: int
    income: float
    credit_score: int
    has_debt: bool

class Output(BaseModel):
    prediction: float
    probability: float

@app.post("/predict", response_model=Output)
def predict(data: Input):
    # Process with model
    return Output(...)
```



Pydantic provides:

- Automatic request validation
- Type conversion
- Clear API documentation
- Response validation

Containerization

Dockerfile Example

```
FROM tiangolo/uvicorn-  
gunicorn-fastapi:python3.11  
COPY requirements.txt .  
RUN pip install -r  
requirements.txt  
COPY . /app
```

Build & Run

```
docker build -t fastapi-model .  
docker run -p 8000:80 fastapi-  
model
```

Benefits

- Consistent environments
- Dependency isolation
- Easy deployment
- Versioning support



Scaling & Production

Concurrency

Use **Gunicorn** workers to handle multiple requests simultaneously

Orchestration

Deploy on **Kubernetes** with Horizontal Pod Autoscaling based on CPU/GPU metrics

Load Balancing

Distribute traffic with Envoy/NGINX or cloud load balancers

Caching

Add Redis layer for caching frequent inference requests



Observability

Monitor your ML services in production to catch issues before they affect users



Request Logging

Use middleware to log requests, responses, and execution times



Prometheus Metrics

Track request rates, latencies, and error rates with prometheus-fastapi-instrumentator



Distributed Tracing

Implement OpenTelemetry to trace requests across services



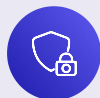
Dashboards

Monitor p95/p99 latency and error rates with Grafana or cloud monitoring



Security & Governance

Protect your ML APIs and ensure compliance with industry standards



TLS Termination

Secure communications with HTTPS via Ingress/Gateway configurations



Authentication

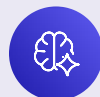
Implement OAuth2/JWT for endpoint authorization

```
@app.post("/predict")
def predict(data: Input, token: str = Depends(oauth2_scheme)):
```



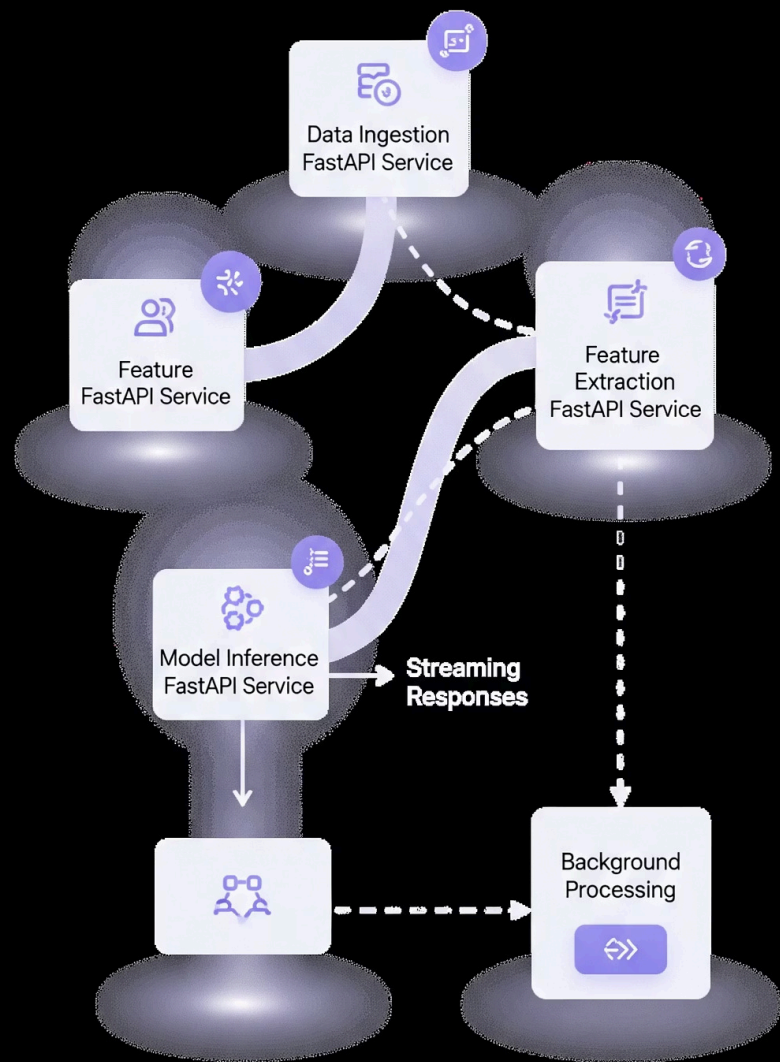
Rate Limiting

Protect resources with API gateway rules or NGINX annotations



Auditability

Log request IDs and model versions for complete traceability



Advanced Patterns

Async Endpoints

```
@app.post("/predict")
async def predict(data: Input):
    result = await
    run_inference(data)
    return result
```

Perfect for I/O-bound operations or calling external services

Streaming Responses

```
@app.post("/generate")
async def generate(prompt: str):
    async def token_generator():
        for token in
            model.generate(prompt):
                yield {"token": token}
    return
    StreamingResponse(token_generat
        or())
```

Ideal for LLMs and generative AI

Other patterns: background tasks for batch jobs, canary/A-B testing via multiple services

Common Pitfalls



Production Configuration

Serving directly with uvicorn instead of using Gunicorn workers

Code Organization

Mixing preprocessing with inference code makes maintenance difficult

Version Control

Not pinning model/library versions leads to drift issues

Performance Testing

Skipping load tests only to discover performance issues in production

Key Takeaways

Simplicity & Speed

FastAPI provides a simple, fast way to wrap ML models as APIs with minimal boilerplate

Python-First

Great fit for **Python teams** and lightweight services without adding complexity

Production-Ready

Containerize and add observability tools for robust production deployments

Enterprise Scale

Scale horizontally with load balancing and Kubernetes for high-demand applications

Start small, iterate quickly, and add complexity only as needed

